Stream Reasoning with Sequencing: Supplementary material

Manolis Pitsikalis and Alexander Artikis

We present first the syntax for defining simple fluents, and second the proofs for our propositions.

1 Defining simple fluents

An event description in $\mathsf{RTEC_s}$ may contain definitions of simple fluents and statically determined fluents. Simple fluents are defined as follows.

Definition 1 (Syntax)**.** *The rules defining* initiatedAt *predicates in* $\mathsf{RTEC_s}$ *have the following syntax:*

$$\begin{aligned} &\mathsf{initiatedAt}(F = V,\ T,) \leftarrow \\ &\quad \mathsf{happensAt}(E_1,\ T), \\ &\quad [[\,[\mathsf{not}]\ \mathsf{happensAt}(E_2,\ T),\ \ldots, \\ &\quad\ \ [\mathsf{not}]\ \mathsf{happensAt}(E_i,\ T), \\ &\quad\ \ [\mathsf{not}]\ \mathsf{holdsAt}(F_1 = V_1,\ T),\ \ldots, \\ &\quad\ \ [\mathsf{not}]\ \mathsf{holdsAt}(F_k = V_k,\ T).\,]] \end{aligned} \tag{1}$$

The first body literal of an initiatedAt *rule is a positive* happensAt *predicate; this is followed by a possibly empty set of positive/negative* happensAt *and* holdsAt *predicates denoted by '*$[[\,]]$*'.* $\mathsf{holdsAt}(F = V,\ T)$ *expresses that fluent* F *has value* V *at a given time-point* T*.* 'not' *expresses negation-by-failure, while '*$[\mathsf{not}]$*' denotes that* 'not' *is optional. All (head and body) predicates are evaluated on the same time-point* T*.* terminatedAt *rules have the same form.*

2 Proofs

Proposition 3 (Monotonic reasoning)**.** *The interval list* $I_\circ$ *computed by means of* $\mathsf{holdsFor}^\bullet(F = V, \langle I_\circ, T_\circ\rangle)$ *at* q_i *will never be revised at a future query-time.*

Proof sketch. Since non-monotonic output can only be introduced via the sequence operator, it suffices to prove the proposition for FVPs defined in terms of this operator. All other operations of the RTEC language produce monotonic output when applied to monotonic input. We proceed by induction on the maximum number of nested sequence operations in the definitions of the input fluents of a sequence operation.

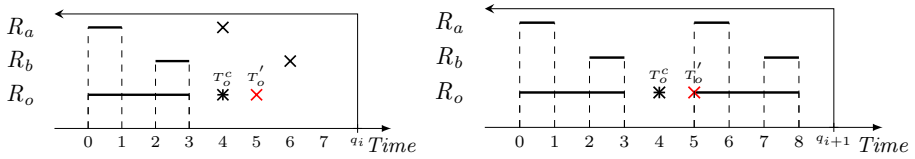
For the base case, consider $\mathsf{seq}(R_a, R_b, R_\circ)$, where $R_a = \langle I_a, T_a \rangle$ and $R_b = \langle I_b, T_b \rangle$ are produced by FVPs whose definitions do not use the sequence operator and do not depend, directly or indirectly, on FVPs that use this operator. Let $R_\circ = \langle I_\circ, T_\circ \rangle$ be the RCT computed at query-time q_i over the narrative $\mathcal{N}_{q_i}$. Since all operations in RTEC are monotonic, the intervals of I_a and I_b preceding their respective cut-off points will not change at any future query-time. Assume, for contradiction, that at some future query-time q_j, where $\mathcal{N}_{q_i} \subseteq \mathcal{N}_{q_j}$, the sequence operation produces an interval $[ts, te)$ before $T_\circ$ which was not produced at q_i. This can only occur if there is an interval $i = [ts, te')$ in I_a for which an adjacent successor $[ts', te)$ may be introduced by the extension $\mathcal{N}_{q_j} \setminus \mathcal{N}_{q_i}$. According to Definitions 12 and 13, the existence of such an interval i sets the cut-off of the sequence operation to a time-point no greater than $s(i) = ts$ i.e., $T_\circ \leq ts$. However, $[ts, te)$ was assumed to belong to the output preceding $T_\circ$, which requires $te \leq T_\circ$ and hence $ts < T_\circ$. This is a contradiction. Therefore, the output interval list of the sequence operation cannot change before $T_\circ$ in the base case.

For the inductive step, assume that the proposition holds for every RCT produced by an FVP whose definition contains at most k nested sequence operations. Consider a sequence operation $\mathsf{seq}(R_a, R_b, R_\circ)$, whose input RCTs are produced by FVPs containing at most k nested sequence operations. Let $R_a = \langle I_a, T_a \rangle$, $R_b = \langle I_b, T_b \rangle$ at q_i. By the induction hypothesis, the intervals of I_a and I_b preceding T_a and T_b respectively, will not change at any future query-time q_j. Assume, for contradiction, that an interval $[ts, te)$ with $ts \leq T_\circ$ is introduced into the output of the sequence operation at some future query-time q_j. Such an interval can be introduced only if an interval $i = [ts, te')$ in I_a acquires an adjacent successor after query time q_i. We distinguish two cases. First, suppose that i already belonged in I_a at q_i. Since its adjacent successor had not yet been observed, i was unresolved at q_i. By Definitions 12 and 13, the existence of such an unresolved interval would cause the output cut-off to be set to $s(i) - 1 = ts - 1$ contradicting our hypothesis that $[ts, te')$ overlaps $T_\circ$, i.e., $ts \leq T_o$. Second, suppose that i did not belong to I_a at q_i, but was introduced at a later query time q_j with $ts > T_a$. Together with the assumption $ts \leq T_\circ$, this yields $T_a < ts \leq T_\circ$. If $T_\circ = T^c_\circ$, then $T^c_\circ > T_a$, by Definition 12, is possible only under case (a). The condition $s(i_b) < nextPosI(T_a)$ in that case ensures that no interval added to I_a after T_a can introduce a new sequence whose start point is at or before $T^c_\circ$. If $T_\circ > T^c_\circ$, then $T_\circ$ is obtained through the extension set E_s. By the definition of E_s, any interval in an extension of I_a that starts at or before $T_\circ$ and does not currently participate in a sequence with an interval of I_b cannot participate in such a sequence under any future extension of I_b. Since i was not present at q_i, it did not participate in a sequence at q_i. Hence, it cannot acquire an adjacent successor at q_j, contradicting our assumption that gives rise to the new output interval $[ts, te)$.

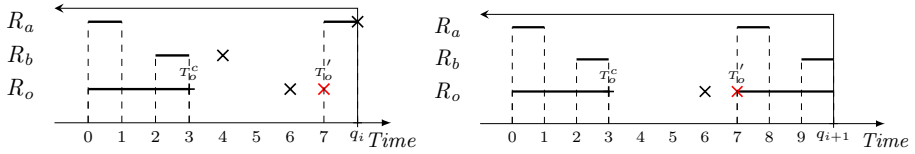
It follows by induction that the interval list produced by any sequence operation remains unchanged up to its computed cut-off point. If the resulting RCT is used as input to another temporal operation, then either this operation is not a sequence operation, in which case monotonic input produces monotonic output, or it is a sequence operation, in which case the inductive argument applies again. Consequently, the interval list $I_\circ$ computed by $\mathsf{holdsFor}^\bullet$ at q_i will never be revised at a future query-time. □

Proposition 4 (Cut-off optimality)**.** *The cut-off point $T_\circ$ computed by means of* $\mathsf{holdsFor}^\bullet(F = V, \langle I_\circ, T_\circ\rangle)$ *at q_i is the latest time-point that guarantees a monotonic output.*

Proof sketch. Given that the computed cut-off points are valid (guarantee monotonicity), to prove cut-off optimality it suffices to show that there cannot be any later cut-off point that guarantees monotonicity. Let us assume that there exists a cut-off $T'_\circ$ point after the the cut-off point $T_\circ$ of the RCT produced by $\mathsf{seq}^\bullet(R_a, R_b, \langle I, T_\circ\rangle)$ at q_i on $\mathcal{N}_{q_i}$. Consider the arrangements for R_a and R_b as illustrated in Figure 1.



(a) Example case 1.



(b) Example case 2.

Figure 1: Example cases where extending the cut-off point by 1 leads to non-monotonic results. $T_\circ$ for $R_\circ$ is illustrated by a tilted cross.

For all the arrangements of Figure 1, we can show that for any $T'_o > T_\circ$ there exists at least one $\mathcal{N}_{q_{i+1}}$ extension of the narrative for which T'_o leads to non-monotonic output. More specifically in Example case 1, if we shift the cut-off point by one time-point, and the next extension of the narrative includes the first permissible interval in R_a ($[5, 6)$) as well as the interval $[7, 8)$ in R_b, then we have non-monotonic output: an interval starts at 5 which is the cut-off point at $T'_\circ$. Similarly in Example case 2, $T_\circ$ is set to 6. This means that the output will not change up to 6 included. If we take as cut-off point the time-point 7, then again, as illustrated in Figure 1 we have non-monotonic output. The same applies for all possible arrangements of R_a and R_b. Similar proofs can be made for the interval manipulation constructs of $\mathsf{RTEC}^{\mathsf{RCT}}_{\mathsf{s}}$. □

Proposition 5 (Soundness and completeness of the bookkeeping policy)**.** *Consider an interval manipulation construct* $\mathsf{iConstruct}^\bullet([R_1, \ldots, R_n], R_\circ)$ *or a sequence operation* $\mathsf{seq}^\bullet(R_1, R_2, R_\circ)$*. According to the bookkeeping policy of Definition 15,* $\mathsf{RTEC}^{\mathsf{RCT}}_{\mathsf{s}}$ *caches all and only the intervals from I_i of R_i ($i \in [1, n]$ or resp. $i \in [1, 2]$) that may be necessary for the computation of the interval manipulation construct or sequence operation at a future query-time.*

Proof. We assume that the domain of the variables of a fluent is known. For the sake of contradiction, assume that an interval $[ts, te) \in I_1$ that ends before $T_\circ$ (i.e., $te < T_\circ$) should have been retained because it could participate in a sequence at a future query.

However, by definition, the output of the sequence operation $\mathsf{seq}^\bullet$ is monotonic up to time $T_\circ$. Therefore, any sequence involving $[ts, te)$ would have had to produce output before $T_\circ$. Retaining $[ts, te)$ would not affect future queries without violating monotonicity. Thus, $[ts, te)$ cannot contribute to any future output and need not be retained. The same argument applies to any interval $[ts, te) \in I_2$ such that $te < T_\circ$. If such an interval ends before $T_\circ$, it too cannot be part of any new sequence in future queries without violating monotonicity. Therefore, it is safe to discard it.

Now, consider an interval $[ts, te) \in I_1$ that overlaps $T_\circ$ (i.e., $ts < T_\circ \leq te$) or starts after $T_\circ$ (i.e., $ts \geq T_\circ$). Since $T_\circ$ is the instant up to which monotonic output is guaranteed, such an interval may still contribute to sequences relevant to future queries. Hence, discarding it would result in a potential loss of information needed for correctness. Therefore, the policy must retain such intervals. An identical argument applies to intervals in I_2. □

Proposition 6 (Correctness of $\mathsf{RTEC}_\mathsf{s}^{\mathsf{RCT}}$)**.** *At each query-time,* $\mathsf{RTEC}_\mathsf{s}^{\mathsf{RCT}}$ *computes the same intervals for* $F = V$ *as* RTEC_s*, up to the cut-off point* $T_\circ$ *of* $\mathsf{holdsFor}^\bullet(F = V, \langle I_\circ, T_\circ \rangle)$*, when the window of* RTEC_s *is large enough so that no sequence is missed.*

Proof. It follows from Propositions 3 and 5. Proposition 3 guarantees that the output of $\mathsf{RTEC}_\mathsf{s}^{\mathsf{RCT}}$ will not change up to the cut-off point, while Proposition 5 guarantees that all required information for stream reasoning is always available. □